

Cluster Usage Documentation

Professor:

Anthony Skjellum

TAs:

1. Shawn Shankar
2. Constanza Zurita

Writing and Compiling MPI Code

1. Writing Your Code:

Write your code in c++, being sure to include mpi.h at the beginning of your code.

```
#include <mpi.h>
```

Remember that your parallel code will begin with MPI_Init(&argc, &argv), and ends with MPI_Finalize().

```
MPI_Init(&argc, &argv);
```

```
MPI_Finalize();
```

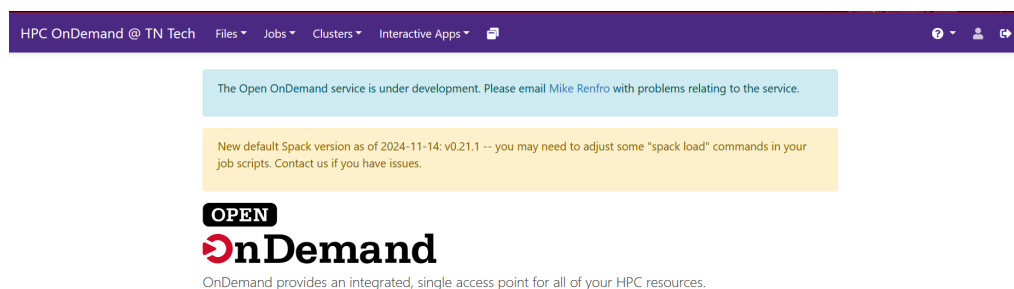
Save your file, and then upload it to the cluster.

2. Uploading Your Code to the Cluster:

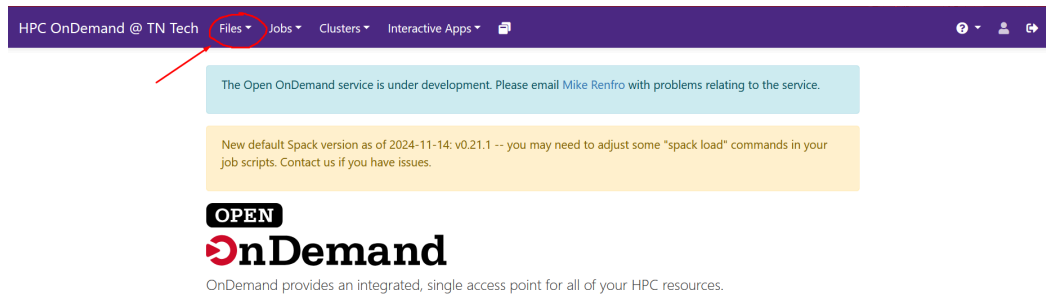
One easy way to do this is to go to the HPC on demand dashboard.

First, enter the following link into your browser: <https://ood.hpc.tnitech.edu>.

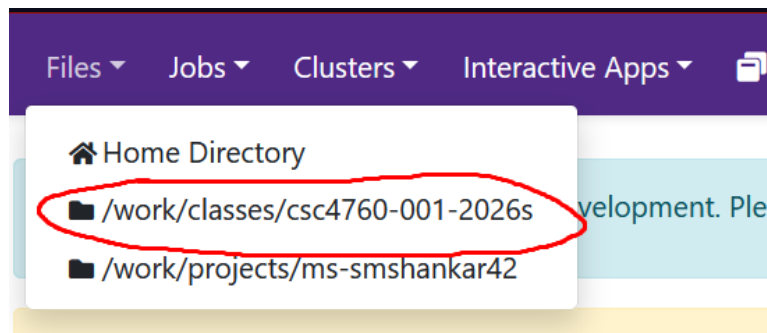
You should see a website that looks like this:



Then, to get to your class folder on the cluster, click on the “Files” button.



From the menu that pops up, click the following folder.



In the following page, scroll down until you see a folder with your tech ID on it, and click on it.

Navigation bar: >_ Open in Terminal | + New File | New Directory | Upload | Download | Copy/Move | Delete

Path: / work / classes / csc4760-001-2026s / | Change directory | Copy path

☐ Show Owner/Mode ☐ Show Dotfiles Filter:

Showing 43 rows - 0 rows selected

Type	Name	Size	Modified at
Folder	alweigant42	-	1/9/2026 8:20:53 PM
Folder	amoya42	-	1/9/2026 8:14:18 PM
Folder	amwheeler43	-	1/9/2026 8:20:53 PM
Folder	arjones49	-	1/9/2026 8:14:18 PM
Folder	askjellum	-	1/9/2026 8:14:18 PM
Folder	asperkins42	-	1/9/2026 8:20:53 PM
Folder	bbmoore42	-	1/9/2026 8:20:53 PM

Once you are in your own folder, you should be able to make folders and files.

[Open in Terminal](#) [+ New File](#) [New Directory](#) [Upload](#) [Download](#) [Copy/Move](#) [Delete](#)

[↑](#) / work / classes / csc4760-001-2026s / smshankar42 / [Change directory](#) [Copy path](#)

☐ Show Owner/Mode ☐ Show Dotfiles Filter:

Showing 4 rows - 0 rows selected

	Type	Name		Size	Modified at
☐	Folder	colorful	⋮	-	2/19/2026 2:39:05 PM
☐	Folder	GoL	⋮	-	2/11/2026 6:24:11 PM
☐	Folder	Ringy_Thingy	⋮	-	2/19/2026 2:50:35 PM
☐	Folder	vector_schmector	⋮	-	2/19/2026 6:38:34 PM

You can also upload any files you have using the upload button.

[Open in Terminal](#) [+ New File](#) [New Directory](#) [Upload](#) [Download](#) [Copy/Move](#) [Delete](#)

[↑](#) / work / classes / csc4760-001-2026s / smshankar42 / [Change directory](#) [Copy path](#)

☐ Show Owner/Mode ☐ Show Dotfiles Filter:

Showing 4 rows - 0 rows selected

	Type	Name		Size	Modified at
☐	Folder	colorful	⋮	-	2/19/2026 2:39:05 PM
☐	Folder	GoL	⋮	-	2/11/2026 6:24:11 PM
☐	Folder	Ringy_Thingy	⋮	-	2/19/2026 2:50:35 PM
☐	Folder	vector_schmector	⋮	-	2/19/2026 6:38:34 PM

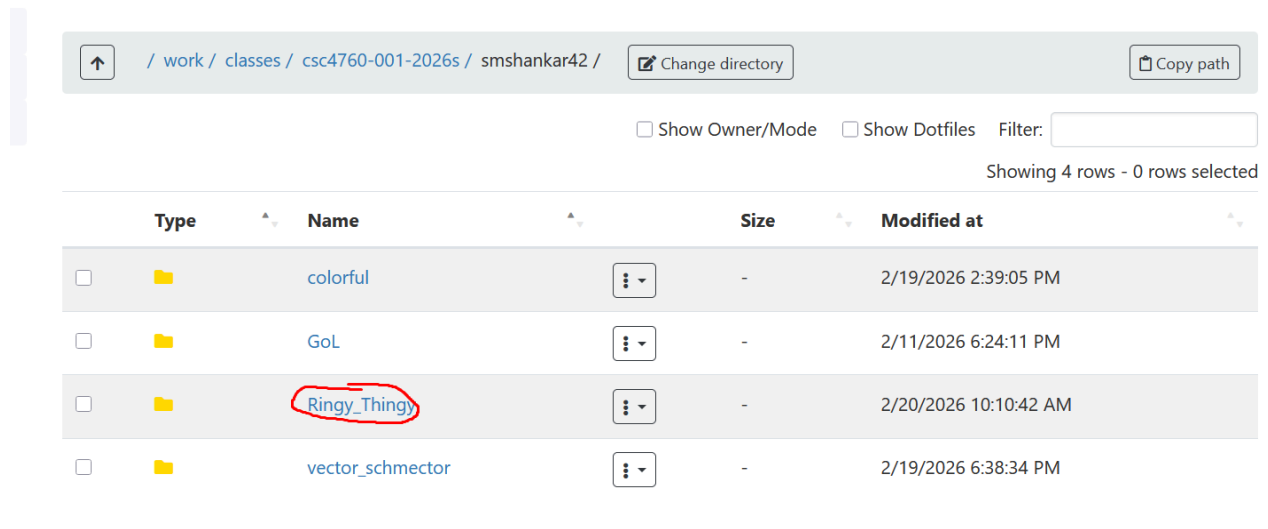
Once you have uploaded your code to your folder in the cluster (or any subfolders within your folder), you can enter the terminal directly by using the “Open in Terminal” button.

3. Compiling Your Code on the Cluster:

For compiling your code, it is best to just log into the debug partition (hpcshell), for quick debugging to see if there are errors in your code that would prevent it from compiling.

From the HPC on demand dashboard, first enter the folder with the file that you want to compile. In this example, the code that I want to compile is called “Problem_1.cpp”, and it is located in the folder “Ringy_Thingy”.

First, I enter the folder where my code is located.



Path: /work/classes/csc4760-001-2026s/smshankar42/

Buttons: Change directory, Copy path

Options: ☐ Show Owner/Mode ☐ Show Dotfiles Filter:

Showing 4 rows - 0 rows selected

Type	Name	Size	Modified at
Folder	colorful	-	2/19/2026 2:39:05 PM
Folder	GoL	-	2/11/2026 6:24:11 PM
Folder	Ringy_Thingy	-	2/20/2026 10:10:42 AM
Folder	vector_schmector	-	2/19/2026 6:38:34 PM

OnDemand version: 2.0.32

See that “Problem_1.cpp” is here.

Type	Name	Size	Modified at
☐	backup.cpp	2.49 KB	2/16/2026 3:12:53 PM
☐	con1_code.cpp	1.84 KB	2/19/2026 2:50:34 PM
☐	init.cpp	1.13 KB	2/16/2026 3:23:49 PM
☐	old.cpp	2.18 KB	2/16/2026 3:07:23 PM
☐	Problem_1.cpp	2.51 KB	2/20/2026 10:12:28 AM
☐	prog	22.6 KB	2/20/2026 10:06:29 AM
☐	prog2	22.7 KB	2/19/2026 2:50:35 PM
☐	ringy_pingy.cpp	2.51 KB	2/16/2026 3:42:58 PM
☐	t	22.6 KB	2/16/2026 3:43:00 PM

Then, enter the terminal from the button at the top.

Type	Name	Size	Modified at
☐	backup.cpp	2.49 KB	2/16/2026 3:12:53 PM
☐	conl_code.cpp	1.84 KB	2/19/2026 2:50:34 PM
☐	init.cpp	1.13 KB	2/16/2026 3:23:49 PM
☐	old.cpp	2.18 KB	2/16/2026 3:07:23 PM
☐	Problem_1.cpp	2.51 KB	2/20/2026 10:12:28 AM
☐	prog	22.6 KB	2/20/2026 10:06:29 AM
☐	prog2	22.7 KB	2/19/2026 2:50:35 PM

You should see a terminal pop up in a new tab like this:

```
Host: 149.149.248.135 Initial directory: /work/classes/csc4760-001-2026s/smshankar42/Ringy_Thingy

Job submission accounts you have access to:
csc4760-001-2026s ms-smshankar42
Group storage folders you have access to:
/work/classes/csc4760-001-2026s /work/projects/ms-smshankar42
[smshankar42@login Ringy_Thingy]$
```

The first step is to log into the debug partition. Use the command “hpcshell -n 8 -A csc4760-001-2026s”.

- Here, the 8 after the “-n” specifies that you want to do 8 processes (these correspond to the ranks in mpi).
- You can modify this number depending on how many ranks you need for the problem; set this to the maximum number of processes you need to use for your MPI code.
 - For example, if a problem makes you test something with 8, 4, and 2 processes, choose 8.
- An aside: “hpcshell” is equivalent to the command “salloc -partition=any-interactive”. When using a different cluster, where “hpcshell” is not a command, you will use “salloc” instead (see the Appendix).

You should get a success message after entering in the command, as seen below. If you get a different message, it probably means that there is either something wrong with the cluster, or there are not enough resources at the moment, and you may have to wait.

```
/work/classes/csc4760-001-2026s /work/projects/ms-smshankar42
[smshankar42@login Ringy_Thingy]$ hpcshell -n 8 -A csc4760-001-2026s
salloc: Granted job allocation 519551
salloc: Nodes gpunode011 are ready for job
[smshankar42@gpunode011 Ringy_Thingy]$
```

Next, load openmpi, using the command “spack load openmpi”. If it succeeds, you will see no message.

```
[smshankar42@gpunode003 Ringy_Thingy]$ spack load openmpi
[smshankar42@gpunode003 Ringy_Thingy]$
```

Then, compile your code using mpic++. You can use the template “mpic++ [name of code file] -o [name of executable]”.

In this example, the code I want to compile is “Problem_1.cpp”, and I output the executable as “p1.x”.

```
[smshankar42@gpunode003 Ringy_Thingy]$ mpic++ Problem_1.cpp -o p1.x
[smshankar42@gpunode003 Ringy_Thingy]$
```

If your code compiles correctly, you won’t see any messages. You can check to see that an output was created, by using ls.

```
[smshankar42@gpunode003 Ringy_Thingy]$ mpic++ Problem_1.cpp -o p1.x
[smshankar42@gpunode003 Ringy_Thingy]$ ls
backup.cpp  bashtest.sh  conl_code.cpp  init.cpp  old.cpp  p1.x  Problem_1.cpp  prog  prog2  ringy_pingy.cpp  schedule.sh  t
[smshankar42@gpunode003 Ringy_Thingy]$
```

If you are ready to test the code, you can move on to the next section in this document.

If you are done, however, BE SURE TO EXIT HPCSHELL. These are shared resources, so be sure to free them up when you are not using them. Just type exit to free up your resources.

```
[smshankar42@gpunode011 Ringy_Thingy]$ exit
exit
salloc: Relinquishing job allocation 519551
salloc: Job allocation 519551 has been revoked.
[smshankar42@login Ringy_Thingy]$
```

4. Using The Debug Partition (hpcshell):

Continuing from the previous section, you can use mpirun to run an executable that you have already built with mpic++.

Using mpirun, you can use the template “mpirun -n [number of processes] ./[name of executable] [program arguments]”.

- The number of processes should be less than or equal to the number that you specified in the beginning with hpcshell/salloc.

- The program arguments are any program arguments/parameters that you specify in the code you wrote at the beginning (These are the arguments that go to `argv[]`).

In this example, I have the compiled executable “p1.x”, and I want to run it with the program argument of 1.

- I want to run it with 8 processes, so I use “-n 8”
- Here the argument passed is 1, which is how many times I pass the number around the ring.

```
[smshankar42@gpunode003 Ringy_Thingy]$ mpirun -n 8 p1.x 1
Running program with N = 1 and 8 tasks
task 0 now sending number: 0 to task 1
task 1 received number: 0 from task 0
sending number: 0 to task 2
task 2 received number: 0 from task 1
sending number: 0 to task 3
task 3 received number: 0 from task 2
sending number: 0 to task 4
task 4 received number: 0 from task 3
sending number: 0 to task 5
task 5 received number: 0 from task 4
sending number: 0 to task 6
task 6 received number: 0 from task 5
sending number: 0 to task 7
task 7 received number: 0 from task 6
sending number: 0 to task 0
task 0 received number: 0 from task 7 now incrementing it to 1
passing complete; number is 1
[smshankar42@gpunode003 Ringy_Thingy]$
```

Suppose that you want to try this problem again, but with a different number of processes.

Let’s say that I want to try this program with 4 processes instead.

The executable “p1.x” is already built, so there is no need to recompile the original code again.

Instead, I just run the exact same command as last time, but I change the number of processes to 4.

- The original command was “mpirun -n 8 p1.x 1”.
- My new command is “mpirun -n 4 p1.x 1”.

```
[smshankar42@gpunode003 Ringy_Thingy]$ mpirun -n 4 p1.x 1
Running program with N = 1 and 4 tasks
task 0 now sending number: 0 to task 1
task 1 received number: 0 from task 0
sending number: 0 to task 2
task 2 received number: 0 from task 1
sending number: 0 to task 3
task 3 received number: 0 from task 2
sending number: 0 to task 0
task 0 received number: 0 from task 3 now incrementing it to 1
passing complete; number is 1
[smshankar42@gpunode003 Ringy_Thingy]$
```

There is no issue when choosing a new number of processes in mpirun, so long as it is equal to or less than the number you specified in hpcshell/salloc.

- An aside – you can run a larger number of processes than you specified in hpc/salloc, with some caveats. See the appendix on oversubscription.

Don't forget to exit when you are done testing.

```
[smshankar42@gpunode011 Ringy_Thingy]$ exit
exit
salloc: Relinquishing job allocation 519559
salloc: Job allocation 519559 has been revoked.
[smshankar42@login Ringy_Thingy]$
```

Scheduling Jobs

Sometimes, you will need to do large jobs that you can't or don't want to sit and watch constantly.

For these jobs, you will want to actually schedule them with slurm.

The first step is to create a slurm script. This is a file that ends with (bash file) “.sh”, and tells the scheduler what program to run, and what parameters to use when doing so. The scheduler will attempt to run the program whenever possible, and return an output to the same folder as the script.

There are several parts of note within the script, for things that you specify:

- The name of the compiled executable you want to run.
- Any argv[] arguments you want to run the program with.
- The upper time limit (how long do you think the program will need to run to reach completion).
- Number of tasks (you can set a default within the script, and then change it when you submit a job).

Here is an example of a slurm script, called “schedule.sh”:


```
Save /work/classes/csc4760-001-2026s/smshankar42/Ringy_Thingy/schedule.sh
1  #!/bin/bash
2
3  #SBATCH --job-name=program1
4  #SBATCH --ntasks=2
5  #SBATCH --cpus-per-task=1
6  #SBATCH --time=00:02:00
7
8  #SBATCH --account=csc4760-001-2026s
9  spack load openmpi
10 export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}
11 mpirun ./p1.x 5
```

Let's walk through each line of the code:

- Line 1 is necessary for the slurm batch script.
- Line 3 specifies the name of the job.
- Line 4 specifies a default number of processes to allocate for the program.
- Line 5 specifies that there will be 1 cpu per task(process) – we are not doing any multithreading in MPI, so this will stay as 1. Here, cpus = cores.
- Line 6 specifies an upper time limit for the program to execute once it is scheduled – try to estimate how long your code will take to run, and aim a bit higher. It is in the format “hours:minutes:seconds”.
- Line 8 specifies the account, just like when you reserve resources using salloc or hpcshell
- Line 9 is using spack to load openmpi, just like in the debug partition.
- Line 10 is irrelevant to the first assignment, but it will be relevant later when we do assignments that require openmp to be used with Kokkos.
- Line 11 calls mpirun with the needed information (what the name of the executable is, any program arguments that need to be passed to argv[] in the program).

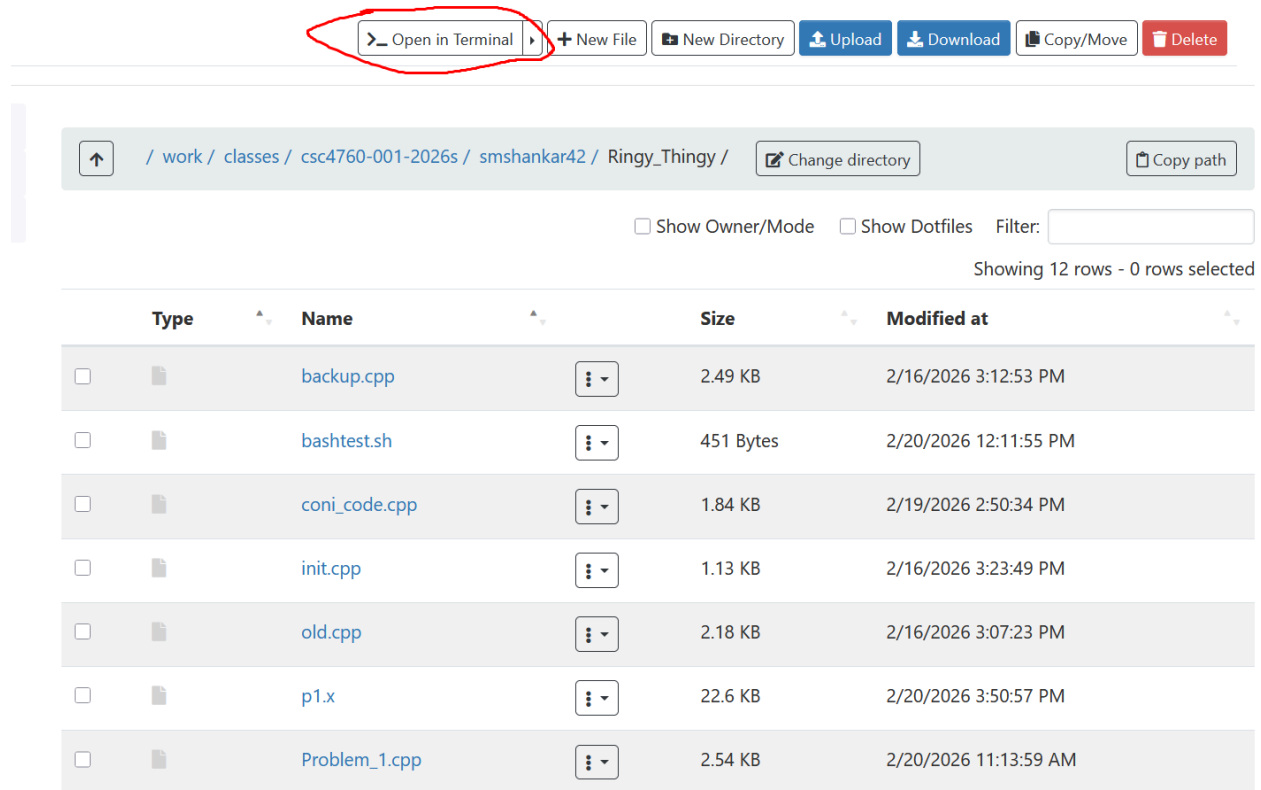
In my specific example, I have set the batch script to run my executable program “p1.x”, with 2 processes, passing the value of 5 to argv[], and giving it an upper time limit of 2 minutes to complete.

Make sure that the batch script is in the same folder as the executable.

Type	Name	Size	Modified at
☐	backup.cpp	2.49 KB	2/16/2026 3:12:53 PM
☐	bashtest.sh	451 Bytes	2/20/2026 12:11:55 PM
☐	conl_code.cpp	1.84 KB	2/19/2026 2:50:34 PM
☐	init.cpp	1.13 KB	2/16/2026 3:23:49 PM
☐	old.cpp	2.18 KB	2/16/2026 3:07:23 PM
☐	p1.x	22.6 KB	2/20/2026 3:50:57 PM
☐	Problem_1.cpp	2.54 KB	2/20/2026 11:13:59 AM
☐	prog	22.6 KB	2/20/2026 10:06:29 AM
☐	prog2	22.7 KB	2/19/2026 2:50:35 PM
☐	ringy_pingy.cpp	2.51 KB	2/16/2026 3:42:58 PM
☐	schedule.sh	227 Bytes	2/20/2026 4:16:09 PM

At this point, I can submit schedule.sh to slurm, and have it execute “p1.x” with the given parameters.

For submitting a job, you don’t have to log into the hpcshell; you can just submit the job from the login node. Just make sure that you are in the right folder.



The screenshot shows the HPC on demand dashboard interface. At the top, there is a navigation bar with several buttons: "Open in Terminal" (highlighted with a red circle), "New File", "New Directory", "Upload", "Download", "Copy/Move", and "Delete". Below this is a breadcrumb navigation path: "/ work / classes / csc4760-001-2026s / smshankar42 / Ringy_Thingy /". There are also buttons for "Change directory" and "Copy path". Below the navigation bar, there are checkboxes for "Show Owner/Mode" and "Show Dotfiles", and a "Filter:" input field. The main content area displays a table of files with columns: Type, Name, Size, and Modified at. The table shows 12 rows, but only 0 rows are selected.

Type	Name	Size	Modified at
☐	backup.cpp	2.49 KB	2/16/2026 3:12:53 PM
☐	bashtest.sh	451 Bytes	2/20/2026 12:11:55 PM
☐	conic_code.cpp	1.84 KB	2/19/2026 2:50:34 PM
☐	init.cpp	1.13 KB	2/16/2026 3:23:49 PM
☐	old.cpp	2.18 KB	2/16/2026 3:07:23 PM
☐	p1.x	22.6 KB	2/20/2026 3:50:57 PM
☐	Problem_1.cpp	2.54 KB	2/20/2026 11:13:59 AM

From there, just use "sbatch [name of slurm batch script]". In my example, I enter "sbatch schedule.sh". This will use all of the parameters that you specified in the slurm script earlier.

```
[smshankar42@login Ringy_Thingy]$ sbatch schedule.sh
Submitted batch job 519706
[smshankar42@login Ringy_Thingy]$
```

You will have to wait a bit, but just use "ls" to periodically check what is in the folder. When you see slurm-xxxxxx.out, that means that your program was executed and slurm returned its output.

```
[smshankar42@login Ringy_Thingy]$ ls
backup.cpp  bashtest.sh  conic_code.cpp  init.cpp  old.cpp  p1.x  Problem_1.cpp  prog  prog2  ringy_pingy.cpp  schedule.sh  slurm-519706.out  t
[smshankar42@login Ringy_Thingy]$
```

Here is what the output looks like – you can either check it out from the terminal, or from the HPC on demand dashboard.

```

1  Running program with N = 5 and 2 tasks
2  task 0 now sending number: 0 to task 1
3  task 0 received number: 0 from task 1 now incrementing it to 1
4  task 0 now sending number: 1 to task 1
5  task 0 received number: 1 from task 1 now incrementing it to 2
6  task 0 now sending number: 2 to task 1
7  task 0 received number: 2 from task 1 now incrementing it to 3
8  task 0 now sending number: 3 to task 1
9  task 0 received number: 3 from task 1 now incrementing it to 4
10 task 0 now sending number: 4 to task 1
11 task 0 received number: 4 from task 1 now incrementing it to 5
12 passing complete; number is 5
13 task 1 received number: 0 from task 0
14 sending number: 0 to task 0
15 task 1 received number: 1 from task 0
16 sending number: 1 to task 0
17 task 1 received number: 2 from task 0
18 sending number: 2 to task 0
19 task 1 received number: 3 from task 0
20 sending number: 3 to task 0
21 task 1 received number: 4 from task 0
22 sending number: 4 to task 0
23

```

As you can see, it ran with the default parameters that were specified in the slurm batch script:

- 2 processes were used
- A value of 5 was passed to argv[]

But what if you wanted to run this with a different number of processes?

To do this, submit the same slurm batch script, but with an additional flag: “--ntasks=4”. This will override the default value specified in the script, and instead run the program with 4 processes instead.

- The original slurm batch script is not changed, it will instead ignore the default value of --ntasks=2 and run the program with --ntasks=4 instead.

```

[smshankar42@login Ringy_Thingy]$ sbatch --ntasks=4 schedule.sh
Submitted batch job 519707
[smshankar42@login Ringy_Thingy]$

```

Once again, just wait for a bit, and then use “ls” to scan the folder for the output file.

```

[smshankar42@login Ringy_Thingy]$ ls
backup.cpp  bashtest.sh  con_i_code.cpp  init.cpp  old.cpp  p1.x  Problem_1.cpp  prog  prog2  ringy_pingy.cpp  schedule.sh  slurm-519706.out  slurm-519707.out  t
[smshankar42@login Ringy_Thingy]$

```

Checking the output file, we can see that the program ran with 4 processes instead of 2.

```
1 Running program with N = 5 and 4 tasks
2 task 0 now sending number: 0 to task 1
3 task 0 received number: 0 from task 3 now incrementing it to 1
4 task 0 now sending number: 1 to task 1
5 task 0 received number: 1 from task 3 now incrementing it to 2
6 task 0 now sending number: 2 to task 1
7 task 0 received number: 2 from task 3 now incrementing it to 3
8 task 0 now sending number: 3 to task 1
9 task 0 received number: 3 from task 3 now incrementing it to 4
10 task 0 now sending number: 4 to task 1
11 task 0 received number: 4 from task 3 now incrementing it to 5
12 passing complete; number is 5
13 task 1 received number: 0 from task 0
14 sending number: 0 to task 2
15 task 1 received number: 1 from task 0
16 sending number: 1 to task 2
17 task 1 received number: 2 from task 0
18 sending number: 2 to task 2
19 task 1 received number: 3 from task 0
20 sending number: 3 to task 2
21 task 1 received number: 4 from task 0
22 sending number: 4 to task 2
23 task 2 received number: 0 from task 1
24 sending number: 0 to task 3
25 task 2 received number: 1 from task 1
26 sending number: 1 to task 3
27 task 2 received number: 2 from task 1
28 sending number: 2 to task 3
29 task 2 received number: 3 from task 1
30 sending number: 3 to task 3
31 task 2 received number: 4 from task 1
32 sending number: 4 to task 3
33 task 3 received number: 0 from task 2
34 sending number: 0 to task 0
35 task 3 received number: 1 from task 2
36 sending number: 1 to task 0
37 task 3 received number: 2 from task 2
38 sending number: 2 to task 0
39 task 3 received number: 3 from task 2
40 sending number: 3 to task 0
41 task 3 received number: 4 from task 2
42 sending number: 4 to task 0
43
```

Appendix

Hpcshell is equivalent to salloc.

```
[smshankar42@login Ringy_Thingy]$ salloc --partition=any-interactive -A csc4760-001-2026s
salloc: Pending job allocation 519674
salloc: job 519674 queued and waiting for resources
salloc: job 519674 has been allocated resources
salloc: Granted job allocation 519674
salloc: Nodes gpunode002 are ready for job
[smshankar42@gpunode002 Ringy_Thingy]$
```

Sometimes you get assigned to a GPU node.

```
/work/classes/csc4760-001-2026s /work/projects/ms-smshankar42
[smshankar42@login Ringy_Thingy]$ hpcshell -n 8 -A csc4760-001-2026s
salloc: Granted job allocation 519551
salloc: Nodes gpunode011 are ready for job
[smshankar42@gpunode011 Ringy_Thingy]$
```

While you may not be using a GPU for a problem, salloc may end up putting you on a GPU node, as seen above. Don't worry about this.

- Oversubscription:

Sometimes, you might want to run more processes than you initially allocated when using hpcshell or salloc. In this case, you can use oversubscription to do so.

In short, oversubscription is running more processes on the same amount of resources, which results in worse performance, and can mess up benchmarking.

- If you are trying to get benchmarks, do not use oversubscription.

In this example, I have allocated 8 processes.

```
[smshankar42@login Ringy_Thingy]$ salloc --partition=any-interactive -n 8 -A csc4760-001-2026s
salloc: Granted job allocation 519699
```

This means that the system has allocated resources to optimally run 8 processes.

If I just attempt to run my executable (p1.x from earlier in the document) with 16 processes, I will see the following message:

```
[smshankar42@gpunode003 Ringy_Thingy]$ mpirun -n 16 p1.x 1
-----
There are not enough slots available in the system to satisfy the 16
slots that were requested by the application:

    p1.x

Either request fewer slots for your application, or make more slots
available for use.

A "slot" is the Open MPI term for an allocatable unit where we can
launch a process. The number of slots available are defined by the
environment in which Open MPI processes are run:

1. Hostfile, via "slots=N" clauses (N defaults to number of
   processor cores if not provided)
2. The --host command line parameter, via a ":N" suffix on the
   hostname (N defaults to 1 if not provided)
3. Resource manager (e.g., SLURM, PBS/Torque, LSF, etc.)
4. If none of a hostfile, the --host command line parameter, or an
   RM is present, Open MPI defaults to the number of processor cores

In all the above cases, if you want Open MPI to default to the number
of hardware threads instead of the number of processor cores, use the
--use-hwthread-cpus option.

Alternatively, you can use the --oversubscribe option to ignore the
number of available slots when deciding the number of processes to
launch.
-----
```

I can still run my executable with 16 processes, if I use the flag "--oversubscribe", as seen below. This is placed before "-n".

Oversubscription will not give parallel speedup, as the resources are not being scaled – effectively, it is running 16 processes on resources meant for 8 processes.

```
[smshankar42@gpunode003 Ringy_Thingy]$ mpirun --oversubscribe -n 16 p1.x 1
Running program with N = 1 and 16 tasks
task 0 now sending number: 0 to task 1
task 1 received number: 0 from task 0
sending number: 0 to task 2
task 2 received number: 0 from task 1
sending number: 0 to task 3
task 3 received number: 0 from task 2
sending number: 0 to task 4
task 4 received number: 0 from task 3
sending number: 0 to task 5
task 5 received number: 0 from task 4
sending number: 0 to task 6
task 6 received number: 0 from task 5
sending number: 0 to task 7
task 7 received number: 0 from task 6
sending number: 0 to task 8
task 8 received number: 0 from task 7
sending number: 0 to task 9
task 9 received number: 0 from task 8
sending number: 0 to task 10
task 10 received number: 0 from task 9
sending number: 0 to task 11
task 11 received number: 0 from task 10
sending number: 0 to task 12
task 12 received number: 0 from task 11
sending number: 0 to task 13
task 13 received number: 0 from task 12
sending number: 0 to task 14
task 14 received number: 0 from task 13
sending number: 0 to task 15
task 15 received number: 0 from task 14
sending number: 0 to task 0
task 0 received number: 0 from task 15 now incrementing it to 1
passing complete; number is 1
[smshankar42@gpunode003 Ringy_Thingy]$
```